

Introduction to C++ & Object Oriented Programming

AIL 7002

Programming Language – C++



- **Compiled language:** runs close to the hardware for speed and control.
- Created by Bjarne Stroustrup; first released in 1985.
- Standardized by ISO (modern standard: C++23 / ISO/IEC 14882).
- **Multi-paradigm:** procedural, object-oriented, and generic programming (templates).
- **Rich standard library (STL):** containers, algorithms, iterators, utilities.
- "a light-weight abstraction programming language [designed] for building and using efficient and elegant abstractions"
- Key idea: C++ balances high performance with powerful abstractions—so you get speed ***and*** expressiveness (with responsibility).

A Typical C++ Program

```
#include <iostream>    // For standard input/output streams: std::cin, std::cout, std::endl
#include <cstdlib>      // For EXIT_SUCCESS
/*
  Program purpose:
  - Read an integer x from the user.
  - Compute the value of (x^2 + 2x + 1).
  - Print the result.

  Note:
  - In C++, `^` is the bitwise XOR operator, NOT exponentiation.
  - So `x^2` does NOT mean x squared.
  - To compute x^2, use `x * x` (or std::pow with care for types).
*/

int main() {
    int x = 0; // User input value
    int y = 0; // Computed result

    // Prompt the user for input.
    std::cout << "Enter a number: " << std::endl;

    // Read x from standard input.
    std::cin >> x;

    // Compute (x^2 + 2x + 1).
    y = x * x + 2 * x + 1;

    // Display the computed result.
    std::cout << "The value of x^2 + 2x + 1 is = " << y << std::endl;

    // Indicate successful program termination (value provided by <cstdlib>).
    return EXIT_SUCCESS;
}
```

Fundamental Types in C++

- **bool**
- **char**
- **short**
- **int**
- **long**
- **float**
- **double**

- **void (absence of any type information)**
- **enum(eration)**
 - `enum WorkingDay {MON, TUE, WED, THU, FRI};`
 - `WorkingDay today = MON; //`

Representing a Student

```
#include <iostream> // For std::cout, std::endl
#include <cstdlib>    // For EXIT_SUCCESS
#include <string>     // For std::string

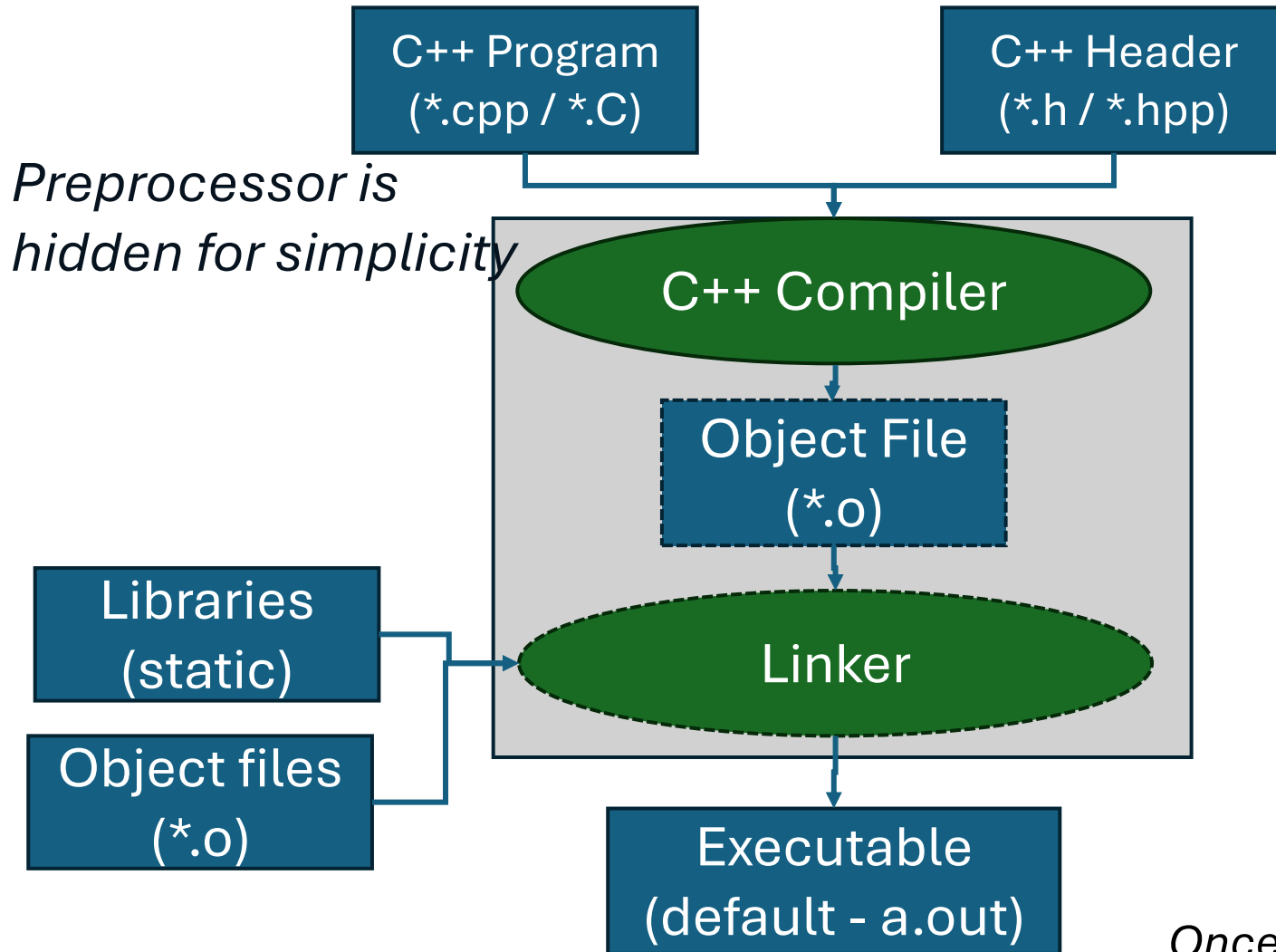
// A simple record type to store basic student information.
struct Student {
    std::string name;      // Student's name
    int batch;             // Batch / cohort number (e.g., 10)
    std::string entrynum;  // Entry number / roll number (stored as string to preserve leading zeros)
};

int main() {
    // Create and initialize a Student object.
    // NOTE: Use normal ASCII double quotes (") in C++ strings, not " " smart quotes.
    Student st1 = {"Veeru", 10, "001"};

    // Print the student's name to standard output.
    std::cout << st1.name << std::endl;

    // Indicate successful program termination.
    return EXIT_SUCCESS;
}
```

A simplified workflow for C++ Programs



> vim test.cpp

> g++ test.cpp

> a.out

Once you run the executable, you will have a process

Student.h

```
#ifndef STUDENT_H
#define STUDENT_H

#include <string>

// A simple Student class demonstrating:
// - private data members (encapsulation)
// - constructors
// - getters/setters
class Student {
private:
    std::string name_;    // Student's name
    int batch_;           // Batch / cohort number
    std::string entrynum_; // Entry number (string keeps leading zeros)

public:
    // Default constructor: initializes members to safe defaults
    Student();

    // Parameterized constructor: initialize all fields
    Student(const std::string& name, int batch, const std::string& entrynum);

    // Getters (do not modify the object)
    std::string getName() const;
    int getBatch() const;
    std::string getEntrynum() const;

    // Setters (modify the object)
    void setName(const std::string& name);
    void setBatch(int batch);
    void setEntrynum(const std::string& entrynum);
};

#endif // STUDENT_H
```

Student.cpp

```
#include "Student.h"

// Default constructor
Student::Student()
    : name_(""), batch_(0), entrynum_("") {}

// Parameterized constructor
Student::Student(const std::string& name, int batch, const std::string& entrynum)
    : name_(name), batch_(batch), entrynum_(entrynum) {}

// Getters
std::string Student::getName() const {
    return name_;
}

int Student::getBatch() const {
    return batch_;
}

std::string Student::getEntrynum() const {
    return entrynum_;
}

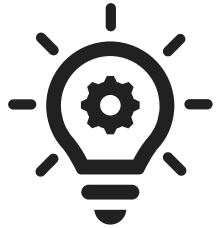
// Setters
void Student::setName(const std::string& name) {
    name_ = name;
}

void Student::setBatch(int batch) {
    batch_ = batch;
}

void Student::setEntrynum(const std::string& entrynum) {
    entrynum_ = entrynum;
}
```

Object Oriented Programming

Object Orientation



Every Object contains

A set of value(s) related to the object

A set of behaviors that can be applied to the object's value (e.g., correct the name, update the batch)



Class

A “template” for objects – values identifiers, methods that define the behavior of the object



Instantiate Objects from Class

Goals of Object-oriented Approach

- **Robustness**

- We want software to be capable of handling unexpected inputs that are not explicitly defined for its application.

- **Adaptability**

- Software needs to be able to evolve over time in response to changing conditions in its environment.

- **Reusability**

- The same code should be usable as a component of different systems in various applications.

Core Ideas of Object-Oriented Approach

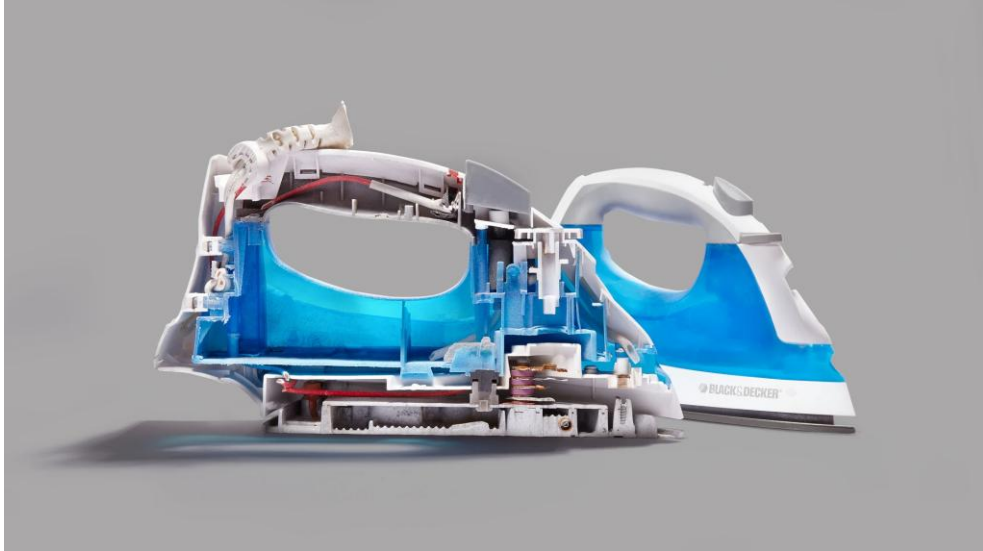
- **Idea 1 : Abstraction**

- We only need to know "what" needs to be the behavior, not "how" it is implemented
 - Abstraction is to distill a system to its most fundamental parts.
- Applying the abstraction paradigm to the design of data structures gives rise to abstract data types (ADTs).
- Abstract Data Types (ADTs) are mathematical models of a data structure
 - Type of data stored
 - Operations supported on them and the parameters of these operations
- An ADT specifies **what** each operation does, but **not how** it does it.
- The collective set of behaviors supported by an ADT is its **public interface**.

Core Ideas of Object-Oriented Approach

- **Idea 2 : Encapsulation**

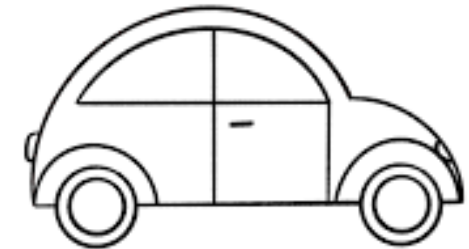
- Reveal only the necessary information, not the internal details of an object
 - Data as well as behavior
- It is true of many every-day objects we use



<https://www.fastcompany.com/90320298/these-oddly-satisfying-photos-reveal-the-inner-workings-of-everyday-objects>



<https://en.comun.app/blog/como-usar-cajeros-automaticos-de-manera-segura-consejos-y-trucos>

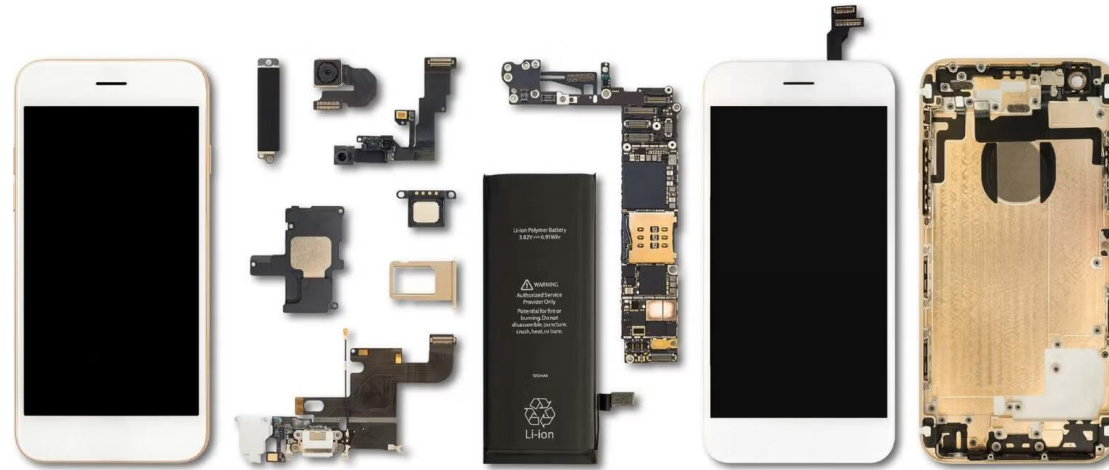


<https://thefairyglitchmother.com/car-drawing-for-kids-how-to-make-it-easy-peasy/>

Core Ideas of Object-Oriented Approach

- **Idea 3 : Modularity**

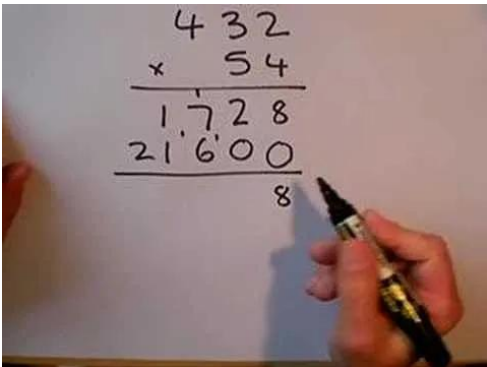
- A large software is divided into **separate** functional units
- Each unit has a well-defined functionality (abstracted appropriately) so that different units can be independently be developed against their functional abstraction



Modular design of an iphone
(from trustedreviews.com)

Abstract Data Type

- The key idea is that a (data)type is characterized by the operations you can perform on it.
- **Number** : you can compare, add, subtract etc.
- **String**: Concatenate and substring
- **Boolean** is something that you can negate ...
- But there is no constraint on **how** any of these operations have to be done



Abstract Data Type of Boolean

- Boolean is an ADT that we would like to build

true: Boolean
false: Boolean

and: Boolean $\times$ Boolean $\rightarrow$ Boolean
or: Boolean $\times$ Boolean $\rightarrow$ Boolean
not: Boolean $\rightarrow$ Boolean

What are the ways in which we can implement Boolean?

t : reference to another object (could be of different type)

T : reference to the object of the ADT we are working with

ADT Operator Classification

- **Creator:** create or build a value of the ADT.
 - Examples: `create()`, `makeStack()`,
 - Operator defn.: $t^* \rightarrow T$ (no T input; builds a fresh T)

Takes zero or more input objects –not necessarily of type T – and builds a new T
- **Producer:** uses existing T s to produce a new T
 - Examples: `union(Set x, Set y)`, `intersect(Set T, Set T)`
 - Operator defn.: $T^+ \times t^* \rightarrow T$

Takes one or more input ADT objects, and zero or more other value/object, and returns a new T

Sometimes the difference between Creator and Producer is blurred

t : reference to
another object (could
be of different type)

T : reference to the
object of the ADT we
are working with

ADT Operator Classification

- **Observer: observe** the ADT without changing it;
return info/value

- Examples: `find(L, x)`, `size(A)`, `isEmpty(A)`
- Operator defn: $T^+ \times t^* \rightarrow (\text{non-T})$

*Takes at least one ADT and an optional element of another kind, and returns a value that could be of any type (except the **original ADT**)*

- **Mutator or Transformer:** take an ADT value and change it or
return an updated version

- Examples: `delete(L, p)`, `push(S, x)`, ...
- Operator defn: $T^+ \times t^* \rightarrow T$

*Takes **at least one ADT**, and an optional element of another kind, returns the ADT that is the result of the change*

A SimpleString ADT in C++

“simple.h”

```
#include <iostream>

class SimpleString {
public:
    char *val;

    static SimpleString* constructSimpleString (bool flag);

    SimpleString* substr (int start, int end);
    int length ();
    char charAt (int i);
};
```

SimpleString Implementation

“simple.cpp”

```
#include "simple.h"
#include <iostream>
#include <cstdlib>
#include <cstring> // strlen, memcpy

using namespace std;

//-----Static methods
SimpleString* SimpleString::constructSimpleString (bool flag) {
    SimpleString *ret = new SimpleString();
    ret->val = flag ? new char[] {'t','r','u','e'} : new char[] {'f','a','l','s','e'};
    return ret;
};

//-----Member methods

int SimpleString::length() {
    return (this->val == nullptr) ? 0 : (int)strlen(this->val);
}

// Keeping your 1-based indexing (charAt(1) returns first char)
char SimpleString::charAt(int i) {
    if (this->val == nullptr) return '\0';
    int n = (int)strlen(this->val);
    if (i < 1 || i > n) return '\0'; // out of bounds
    return this->val[i - 1];
}
```

SimpleString Implementation

“simple.cpp”

```
// Returns substring from [start, end) i.e., start inclusive, end exclusive (0-based)
SimpleString* SimpleString::substr(int start, int end) {
    // Basic sanity
    if (this->val == nullptr) return nullptr;

    int n = (int)strlen(this->val);

    // Clamp to valid range
    if (start < 0) start = 0;
    if (end > n) end = n;

    // Empty or invalid interval => empty string
    if (start >= end) {
        SimpleString* ret = new SimpleString();
        ret->val = new char[1];
        ret->val[0] = '\0';
        return ret;
    }

    int len = end - start;

    SimpleString* ret = new SimpleString();
    ret->val = new char[len + 1]; // +1 for null terminator
    memcpy(ret->val, this->val + start, len); // copy substring bytes
    ret->val[len] = '\0';

    return ret;
}
```

Running SimpleString

“main-simple.cpp”

```
#include <iostream>
#include "simple.h"
```

```
using namespace std;
```

```
int main() {
    cout << "Starting" << endl;

    SimpleString* ss = SimpleString::constructSimpleString(false);
    if (ss == nullptr || ss->val == nullptr) {
        cout << "constructSimpleString returned null/empty" << endl;
        return 1;
    }

    cout << "SimpleString = " << ss->val << endl;
    cout << "SimpleString length " << ss->length() << endl;
    cout << "SimpleString char at 2 is " << ss->charAt(2) << endl;

    // Example use of substr (0-based, end exclusive):
    SimpleString* t = ss->substr(1, 4);
    if (t != nullptr && t->val != nullptr) {
        cout << "Substring [1,4) = " << t->val << endl;
    }

    return 0;
}
```

```
/courses/COL106/c++-code
/courses/COL106/c++-code
```

```
g++ main-simple.cpp simple.cpp -o simple
./simple
```

```
Starting
SimpleString = false
SimpleString length 5
SimpleString char at 2 is a
Substring [1,4) = als
```